

Procesamiento Multimedia

Q3 2025

RtAudio C++ Library

Ing. Iván de Jesús Deras Tábor

Presentación preparada con ayuda IA

Objetivos de la sesión

- Comprender qué son las bibliotecas de audio en tiempo real
- Conocer RtAudio como biblioteca C++ multiplataforma
- Implementar ejemplos prácticos

¿Qué son las bibliotecas de audio en tiempo real?

Definición

Software que permite **procesar audio con latencia mínima**, ideal para:

- Aplicaciones musicales interactivas
- Efectos de audio en tiempo real
- Síntesis de sonido
- Grabación y reproducción profesional

Características clave

- **Baja latencia** (< 10ms idealmente)
- **Callback-based processing**
- **Gestión de buffers** automática
- **Abstracción del hardware** de audio

RtAudio: Biblioteca C++ multiplataforma

¿Qué es RtAudio?

RtAudio es una biblioteca de clases C++ que proporciona una API común para entrada y salida de audio en tiempo real a través de Linux (ALSA, JACK, PulseAudio y OSS), macOS (CoreAudio y JACK), y Windows (DirectSound, ASIO y WASAPI).

Objetivos de diseño

- **Diseño orientado a objetos** en C++
- **API simple y común** entre plataformas
- **Solo un archivo fuente y header** para fácil inclusión
- **Soporte multi-API** simultáneo
- **Conversión automática** de formatos de datos

Arquitectura de RtAudio

Concepto de streams de audio

- **Playback stream:** Salida de audio (reproducción)
- **Capture stream:** Entrada de audio (grabación)
- **Duplex stream:** Entrada y salida simultáneas

Componentes principales

- **AudioContext:** Maneja el contexto de audio
- **Stream parameters:** Configuración del stream
- **Callback function:** Función de procesamiento
- **Device enumeration:** Listado de dispositivos disponibles

Formatos de datos en RtAudio

Tipos de datos soportados

```
// Enteros con signo  
RTAUDIO_SINT8      // 8-bit signed integer  
RTAUDIO_SINT16     // 16-bit signed integer  
RTAUDIO_SINT24     // 24-bit signed integer  
RTAUDIO_SINT32     // 32-bit signed integer  
  
// Punto flotante (normalizado entre -1.0 y 1.0)  
RTAUDIO_FLOAT32    // 32-bit float  
RTAUDIO_FLOAT64    // 64-bit double
```

Orden de bytes

Los datos de audio siempre se asumen en el orden de bytes del host. Las rutinas internas se encargan automáticamente del intercambio de bytes entre el formato del host y la tarjeta de sonido.

Estructura básica de RtAudio

Pasos fundamentales

1. Crear instancia de RtAudio
2. Configurar parámetros del stream
3. Definir función callback
4. Abrir el stream
5. Iniciar procesamiento
6. Cerrar y limpiar

Código básico: Generador de tono

```
#include "RtAudio.h"
#include <iostream>
#include <cmath>

// Estructura para datos del callback
struct CallbackData {
    double frequency;
    double phase;
    double sampleRate;
};

// Función callback para generar tono sinusoidal
int callback(void *outputBuffer, void *inputBuffer,
            unsigned int nBufferFrames, double streamTime,
            RtAudioStreamStatus status, void *userData) {

    CallbackData *data = (CallbackData*)userData;
    double *buffer = (double*)outputBuffer;

    double phaseIncrement = 2.0 * M_PI * data->frequency / data->sampleRate;

    for (unsigned int i = 0; i < nBufferFrames; i++) {
        buffer[i] = 0.3 * sin(data->phase); // Amplitud 0.3
        data->phase += phaseIncrement;
        if (data->phase >= 2.0 * M_PI) data->phase -= 2.0 * M_PI;
    }

    return 0;
}
```

Código básico: Configuración del stream

```
int main() {
    RtAudio dac;

    // Verificar dispositivos disponibles
    std::vector<unsigned int> deviceIds = dac.getDeviceIds();
    if (deviceIds.size() < 1) {
        std::cout << "No hay dispositivos de audio!" << std::endl;
        return -1;
    }

    // Configurar parámetros de salida
    RtAudio::StreamParameters parameters;
    parameters.deviceId = dac.getDefaultOutputDevice();
    parameters.nChannels = 1; // Mono
    parameters.firstChannel = 0;

    unsigned int sampleRate = 44100;
    unsigned int bufferFrames = 256;

    CallbackData data;
    data.frequency = 440.0; // La (A4)
    data.phase = 0.0;
    data.sampleRate = sampleRate;

    // Abrir stream
    try {
        dac.openStream(&parameters, NULL, RTAUDIO_FLOAT64,
                      sampleRate, &bufferFrames, &callback, &data);
        dac.startStream();

        std::cout << "Presiona Enter para detener..." << std::endl;
        std::cin.get();

        dac.stopStream();
    } catch (RtAudioError& e) {
        std::cout << "Error: " << e.getMessage() << std::endl;
    }

    if (dac.isStreamOpen()) dac.closeStream();
    return 0;
}
```

Ejemplo: Grabación de audio

```
// Estructura para datos de grabación
struct InputData {
    std::vector<double> buffer;
    unsigned int frameCounter;
    unsigned int totalFrames;
    unsigned int channels;
};

// Callback para captura de audio
int recordCallback(void *outputBuffer, void *inputBuffer,
    unsigned int nBufferFrames, double streamTime,
    RtAudioStreamStatus status, void *userData) {

    InputData *data = (InputData*)userData;
    double *input = (double*)inputBuffer;

    if (status) std::cout << "Stream overflow detected!" << std::endl;

    // Copiar datos de entrada al buffer
    for (unsigned int i = 0; i < nBufferFrames * data->channels; i++) {
        if (data->frameCounter * data->channels + i < data->buffer.size()) {
            data->buffer[data->frameCounter * data->channels + i] = input[i];
        }
    }

    data->frameCounter += nBufferFrames;

    // Parar cuando se alcance el tiempo deseado
    if (data->frameCounter >= data->totalFrames) return 2;

    return 0;
}
```

Opciones avanzadas de RtAudio

StreamOptions para optimización

```
RtAudio::StreamOptions options;
```

```
// Minimizar latencia
```

```
options.flags = RTAUDIO_MINIMIZE_LATENCY;
```

```
// Uso exclusivo del dispositivo
```

```
options.flags |= RTAUDIO_HOG_DEVICE;
```

```
// Scheduling en tiempo real
```

```
options.flags |= RTAUDIO_SCHEDULE_REALTIME;
```

```
options.priority = 1;
```

```
// Número de buffers (afecta latencia)
```

```
options.numberOfBuffers = 2; // Mínimo para la mayoría de APIs
```

Ciertos APIs de audio ofrecen parámetros que influyen en la latencia de E/S del stream. Por defecto, RtAudio intentará configurar estos parámetros internamente para un rendimiento robusto, aunque posiblemente a expensas del rendimiento del stream.

Recursos y documentación

RtAudio

- **GitHub:** <https://github.com/thestk/rtaudio>
- **Documentación:** <http://www.music.mcgill.ca/~gary/rtaudio/>
- **Ejemplos:** Carpeta `/tests` en el repositorio

Libros recomendados

- “Designing Audio Objects” - Andy Farnell
- “The Computer Music Tutorial” - Curtis Roads

Ejercicios prácticos

Ejercicio 1: Piano Virtual

Implementar un piano virtual con: 1. 12 teclas (octava cromática)
2. Diferentes tipos de onda 3. Control de volumen 4.
Visualización en tiempo real

Ejercicio 2: Grabador Simple

Crear un grabador simple que: 1. Capture audio del micrófono 2.
Guarde a archivo WAV 3. Reproduzca lo grabado

Consideraciones de rendimiento

- **Callback thread:** No hacer I/O ni memory allocation
- **Lock-free programming:** Para comunicación entre threads
- **Buffer management:** Precalculando todo lo posible

// MAL: Allocation en callback

```
int callback(...) {  
    std::vector<double> temp(bufferSize); // ¡NO!  
    return 0;  
}
```

// BIEN: Pre-allocated

```
struct CallbackData {  
    std::vector<double> workBuffer; // Pre-allocated  
};
```

¡Gracias por su atención!